

UNIT IV — Lab Experiments 25–34: Multimodal Generative AI Applications

Name: Joanathan Packia Singh | **Reg. No.:** 192472229 | **Course:** CSA6502

This notebook covers all 10 UNIT IV lab tasks in order. Each lab is self-contained but reuses the shared Groq client / embedding model set up once at the top, the same pattern as the UNIT III (RAG) notebook.

Before you run this: your Groq API key should already be saved as a Colab secret named `groq` (left sidebar → key icon). `LLM_MODEL` below is set to `openai/gpt-oss-120b` since Groq retired `llama-3.3-70b-versatile` and `llama-3.1-8b-instant`.

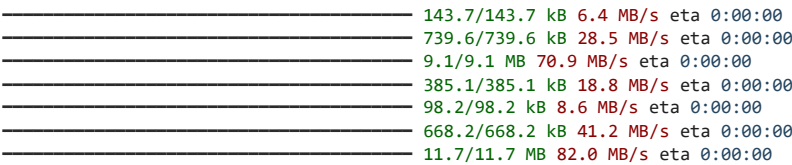
Recommended: for Labs 27–28 (image generation), switch the runtime to a GPU before running: `Runtime → Change runtime type → Hardware accelerator → GPU (T4/L4)`. Everything else in this notebook runs fine on CPU.

Lab	Task
25	College-FAQ chatbot using a pre-trained LLM
26	Technical-support chatbot using NLP preprocessing + an LLM
27	Text-to-image generation from an engineering prompt
28	Multiple images from varied prompts, compared side by side
29	Speech-to-Text for a spoken engineering query
30	Text-to-Speech for engineering text
31	Long-document summarization (map-reduce)
32	English → Indian language machine translation
33	AI resume screening ranked against a job description
34	Research assistant: overview + keywords + summary

Setup — install packages and load shared resources

Run this once before any lab below.

```
!pip install -q --upgrade groq sentence-transformers scikit-learn pypdf gTTS diffusers accelerate transformers sentencepiece langchain-torch
# torch is installed WITHOUT --upgrade on purpose: Colab already ships a GPU-matched build.
# This line only fills it in if it's somehow missing, instead of overwriting it with a
# generic wheel that could lose CUDA support.
!pip install -q torch
```



ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the source of wanda 0.28.1 requires click>=8.2.0, but you have click 8.1.8 which is incompatible.
spacy 3.8.15 requires click<9.0.0,>=8.2.1, but you have click 8.1.8 which is incompatible.

```
import torch
from google.colab import userdata
from groq import Groq
from sentence_transformers import SentenceTransformer, util
import numpy as np

GROQ_API_KEY = userdata.get('groq')
if not GROQ_API_KEY:
    raise RuntimeError(
        "No Groq key found. In Colab: left sidebar -> key icon -> 'Add new secret' -> "
        "name it 'groq' -> paste your Groq API key -> toggle notebook access on."
    )

groq_client = Groq(api_key=GROQ_API_KEY)
# llama-3.3-70b-versatile and llama-3.1-8b-instant were both retired by Groq -> use gpt-oss-120b
LLM_MODEL = "openai/gpt-oss-120b"

def ask_llm(prompt, model=LLM_MODEL, temperature=0.3):
    """Simple helper: send a prompt to Groq and return the text answer."""
    response = groq_client.chat.completions.create(
        model=model,
        messages=[{"role": "user", "content": prompt}],
        temperature=temperature,
    )
    return response.choices[0].message.content

embedder = SentenceTransformer('all-MiniLM-L6-v2') # used in Lab 33

device = "cuda" if torch.cuda.is_available() else "cpu"
```

```

print(f"Groq client ready (model: {LLM_MODEL}). Embedding model ready. Image-gen device: {device}.")
if device == "cpu":
    print("No GPU detected -> Labs 27-28 (image generation) will be slow (roughly a minute or")
    print("more per image). Runtime -> Change runtime type -> Hardware accelerator -> GPU, then")
    print("re-run this cell if you want faster generation.")

```

```

/usr/local/lib/python3.13/dist-packages/huggingface_hub/utils/_auth.py:112: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher
modules.json: 100%                               349/349 [00:00<00:00, 6.87kB/s]

config_sentence_transformers.json: 100%                               116/116 [00:00<00:00, 2.55kB/s]

README.md: 100%                               10.5k/10.5k [00:00<00:00, 376kB/s]

sentence_bert_config.json: 100%                               53.0/53.0 [00:00<00:00, 1.22kB/s]

config.json: 100%                               612/612 [00:00<00:00, 28.7kB/s]

model.safetensors: 100%                               90.9M/90.9M [00:01<00:00, 60.6MB/s]

Loading weights: 100%                               103/103 [00:00<00:00, 4106.67it/s]

tokenizer_config.json: 100%                               350/350 [00:00<00:00, 12.0kB/s]

vocab.txt: 100%                               232k/232k [00:00<00:00, 3.30MB/s]

tokenizer.json: 100%                               466k/466k [00:00<00:00, 9.90MB/s]

special_tokens_map.json: 100%                               112/112 [00:00<00:00, 3.80kB/s]

config.json: 100%                               190/190 [00:00<00:00, 7.15kB/s]
Groq client ready (model: openai/gpt-oss-120b). Embedding model ready. Image-gen device: cuda.

```

Lab 25 — AI Chatbot for Engineering College Student Queries

Goal: Use a pre-trained language model (Groq LLM) to answer student queries about an engineering college, grounded in a short block of college information passed in the prompt.

```

college_info = """
College: SIMATS Engineering (Saveetha School of Engineering)
- Offers B.E./B.Tech programs including CSE, CSE (AI & ML), Mechanical, Civil, and ECE.
- The academic year is split into odd/even semesters, with continuous internal assessment (CIA) plus end-semester exams.
- Final-year students take up a capstone/major project, submitted as a project report with a viva-voce evaluation.
- The campus has department-specific labs, a central library, and supports NPTEL-aligned certification programs alongside the regular curriculum.
- A placement cell coordinates internships and campus recruitment drives with partner companies.
(Swap this block out for your own college's real FAQ facts to reuse this chatbot elsewhere.)
"""

def college_chatbot(student_question):
    system_context = f"""You are a helpful assistant for students at an engineering college.
    Answer using the college information below whenever it's relevant. If the question falls
    outside this information (e.g. it's about a different college, or a topic not covered),
    say so honestly instead of guessing.

    College information:
    {college_info}
    """
    prompt = f"{system_context}\nStudent question: {student_question}\nAnswer:"
    return ask_llm(prompt)

print(college_chatbot("When do students usually take up their capstone/major project?"))
print()
print(college_chatbot("Does this college offer a Bachelor's degree in Fine Arts?"))

```

At SIMATS Engineering, the capstone (or major) project is taken up **during the final year of the B.E./B.Tech program**.

- **When:** Usually in the **7th semester (odd semester)** and the work is completed and submitted by the end of the **8th semester (even semester)**.
 - **What you do:** You'll work on a comprehensive project under the guidance of a faculty supervisor, prepare a detailed project report, a presentation, and defend it.

So, if you're in your final year, you can expect to start your capstone project at the beginning of the 7th semester and finish it before the end of the 8th semester.

No. SIMATS Engineering (Saveetha School of Engineering) does **not** offer a Bachelor's degree in Fine Arts. The college's undergraduate programs are:

- Computer Science & Engineering (CSE)
- Computer Science & Engineering – AI & ML
- Mechanical Engineering

- Civil Engineering
- Electronics & Communication Engineering (ECE)

If you're interested in a Fine Arts program, you would need to look at a different institution that offers arts or humanities degrees.

✓ Lab 26 — Engineering Technical-Support Chatbot Using NLP Techniques

Goal: Before handing a question to the LLM, run a lightweight NLP preprocessing step — tokenization and stopwords filtering — to pull out the key technical terms, and use them to focus the LLM's answer. This is the actual "NLP technique" the task asks for, not just prompting.

```
import re

STOPWORDS = set("""a an the is are was were be been being of to in on for with and or
but if then else this that these those it its as at by from into about over under again
further do does did doing you your yours i we our ours they them their my show why how
what when where""").split())

def extract_technical_keywords(text, max_keywords=6):
    words = re.findall(r"[a-zA-Z][a-zA-Z\-\-]+", text.lower())
    candidates = [w for w in words if w not in STOPWORDS and len(w) > 3]
    seen, unique_keywords = set(), []
    for w in candidates:
        if w not in seen:
            seen.add(w)
            unique_keywords.append(w)
    return unique_keywords[:max_keywords]

def technical_support_chatbot(query):
    keywords = extract_technical_keywords(query)
    prompt = f"""A student asked a technical engineering question. The key technical terms
detected in the question are: {", ".join(keywords)}.

Question: {query}



Give a clear, step-by-step technical answer (3-5 steps or points) that focuses specifically
on those key terms.



Answer: """


```

```
        return keywords, ask_llm(prompt)

query = "Why does my op-amp circuit show clipping distortion at high gain?"
keywords, answer = technical_support_chatbot(query)
print("Detected keywords:", keywords)
print("\nAnswer:\n", answer)
```

```
Detected keywords: ['op-amp', 'circuit', 'clipping', 'distortion', 'high', 'gain']

Answer:
**Why an op-amp circuit starts to clip and sound “distorted” when you crank up the gain**

1. **Output-voltage swing is limited by the supply rails**
  - An op-amp can only drive its output up to a few hundred millivolts (or a few volts for some devices) below the positive supply and ab
  - When the closed-loop gain is high, even a modest input signal is amplified to a voltage that tries to exceed those limits, so the out

2. **Input common-mode range may be exceeded**
  - Many op-amps cannot accept input voltages that get too close to the supply rails. With high gain, the differential input voltage need

3. **Slew-rate and bandwidth constraints**
  - The **gain-bandwidth product (GBW)** of the device is fixed. If you set a very high closed-loop gain, the usable bandwidth shrinks (f
  - At the remaining bandwidth, the op-amp must change its output voltage faster (higher slew rate). If the required dV/dt exceeds the op

4. **Amplified offsets, noise, and non-idealities**
  - Any input offset voltage, bias currents, or noise are also multiplied by the closed-loop gain. At high gain they can push the output

5. **Power-supply headroom and loading**
  - Heavy load resistance or a low-impedance source can pull the output toward the rails, reducing the effective headroom. With high gain

**What to do about it**

- Choose an op-amp with rail-to-rail input and output and a higher supply voltage.
- Reduce the closed-loop gain and add additional amplification stages if you need large overall gain.
- Verify that the required bandwidth is within the op-amp’s GBW and that the slew-rate spec exceeds the maximum dV/dt of your signal.
- Keep input offsets and bias currents low (select a precision op-amp) and provide proper biasing and decoupling to maintain headroom.

By addressing these five factors you can eliminate the clipping that appears when the circuit is set to a very high gain.
```

✓ Lab 27 — Text-to-Image Generation for an Engineering Prompt

Goal: Use a pre-trained text-to-image diffusion model (`stabilityai/sd-turbo`, a small distilled model that generates in 1-4 steps) to turn an engineering text prompt into an image. `sd-turbo` needs `guidance_scale=0.0` and very few inference steps — that's a quirk of how it was distilled, not a bug.

```
from diffusers import AutoPipelineForText2Image
```

```
image_pipe = AutoPipelineForText2Image.from_pretrained(
    "stabilityai/sd-turbo",
    torch_dtype=torch.float16 if device == "cuda" else torch.float32,
)
image_pipe = image_pipe.to(device)
```

```
prompt = "a modern steel suspension bridge over a river at sunset, engineering blueprint style, highly detailed"
```

```
image = image_pipe(prompt=prompt, num_inference_steps=2, guidance_scale=0.0).images[0]
image.save("/content/bridge.png")
image
```

Flax classes are deprecated and will be removed in Diffusers v0.40.0. We recommend migrating to PyTorch classes or pinning your version of Flax classes are deprecated and will be removed in Diffusers v0.40.0. We recommend migrating to PyTorch classes or pinning your version of [transformers] `Siglip2ImageProcessorFast` is deprecated. The `Fast` suffix for image processors has been removed; use `Siglip2ImageProces /usr/local/lib/python3.13/dist-packages/huggingface\_hub/utils/\_validators.py:205: UserWarning: The `local_dir_use_symlinks` argument is de warnings.warn(
model_index.json: 100% 616/616 [00:00<00:00, 25.1kB/s]
Download complete: 100% 5.16G/5.16G [03:59<00:00, 140MB/s]
Fetching 12 files: 100% 12/12 [00:58<00:00, 5.36s/it]
Loading pipeline components...: 100% 5/5 [00:27<00:00, 5.23s/it]
Loading weights: 100% 372/372 [00:05<00:00, 62.16it/s]
You have disabled the safety checker for <class 'diffusers.pipelines.stable_diffusion.pipeline_stable_diffusion.StableDiffusionPipeline'>
100% 2/2 [00:01<00:00, 1.50s/it]



Lab 28 — Multiple Images From Different Prompts, Compared

Goal: Generate several images from related prompts and observe how changing the prompt wording changes the output — here, keeping the subject fixed and only varying the style keyword.

```
import matplotlib.pyplot as plt

prompts = [
    "a humanoid robot arm assembling a car on a factory line, photorealistic",
    "a humanoid robot arm assembling a car on a factory line, cartoon illustration style",
    "a humanoid robot arm assembling a car on a factory line, blueprint sketch style",
]

images = [image_pipe(prompt=p, num_inference_steps=2, guidance_scale=0.0).images[0] for p in prompts]

fig, axes = plt.subplots(1, len(images), figsize=(15, 5))
for ax, img, p in zip(axes, images, prompts):
    ax.imshow(img)
    ax.set_title(p.split(",")[-1].strip(), fontsize=10)
    ax.axis("off")
plt.tight_layout()
plt.show()

print("Observation: keeping the subject ('robot arm assembling a car') fixed and only")
print("swapping the trailing style keyword ('photorealistic' vs 'cartoon' vs 'blueprint')")
```

```
print("sketch') changes rendering style far more than it changes composition -- the model")
print("treats trailing style words as strong conditioning applied on top of the same scene.")
```

100% 2/2 [00:00<00:00, 18.86it/s]

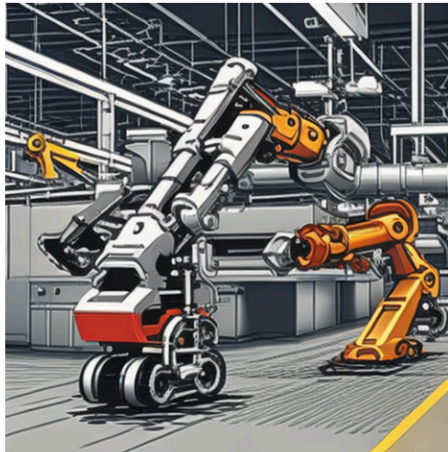
100% 2/2 [00:00<00:00, 19.07it/s]

100% 2/2 [00:00<00:00, 19.34it/s]

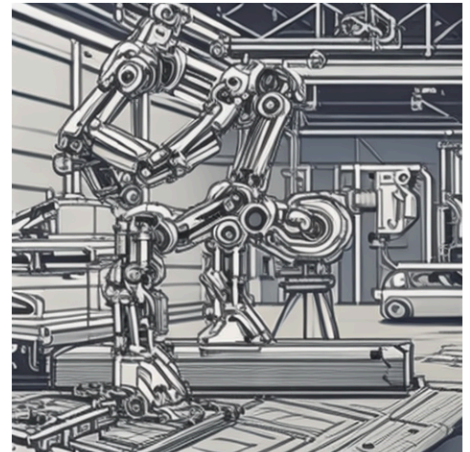
photorealistic



cartoon illustration style



blueprint sketch style



Observation: keeping the subject ('robot arm assembling a car') fixed and only swapping the trailing style keyword ('photorealistic' vs 'cartoon' vs 'blueprint sketch') changes rendering style far more than it changes composition -- the model treats trailing style words as strong conditioning applied on top of the same scene.

Lab 29 — Speech-to-Text for an Engineering Query

Goal: Use a pre-trained ASR model (`openai/whisper-base` via the `transformers` pipeline) to transcribe a spoken engineering question into text.

Run the cell, then either upload your own short audio clip (wav/mp3/m4a) when prompted, or click **Cancel** on the upload dialog — a sample spoken query is auto-generated with gTTS so the lab still runs end-to-end without your own recording.

```
from google.colab import files
from transformers import pipeline as hf_pipeline

asr = hf_pipeline("automatic-speech-recognition", model="openai/whisper-base", device=0 if device == "cuda" else -1)

print("Upload an audio file with a spoken engineering question, or click Cancel to use a sample clip.")
uploaded = files.upload()

if uploaded:
    audio_path = list(uploaded.keys())[0]
else:
    from gtts import gTTS
    audio_path = "/content/sample_query.mp3"
    gTTS("What is the maximum load a steel truss bridge can safely carry?").save(audio_path)
    print(f"No file uploaded -- using auto-generated sample audio: {audio_path}")

result = asr(audio_path)
print("\nTranscribed text:", result["text"])
```


config.json: 100%	1.98k/1.98k [00:00<00:00, 172kB/s]
model.safetensors: 100%	290M/290M [00:01<00:00, 179MB/s]
Loading weights: 100%	245/245 [00:00<00:00, 3585.91it/s]
generation_config.json: 100%	3.81k/3.81k [00:00<00:00, 364kB/s]
tokenizer_config.json: 100%	283k/283k [00:00<00:00, 24.1MB/s]
vocab.json: 100%	836k/836k [00:00<00:00, 19.3MB/s]
tokenizer.json: 100%	2.48M/2.48M [00:00<00:00, 49.2MB/s]
merges.txt: 100%	494k/494k [00:00<00:00, 18.7MB/s]
normalizer.json: 100%	52.7k/52.7k [00:00<00:00, 3.16MB/s]
added_tokens.json: 100%	34.6k/34.6k [00:00<00:00, 1.85MB/s]
special_tokens_map.json: 100%	2.19k/2.19k [00:00<00:00, 146kB/s]
preprocessor_config.json: 100%	185k/185k [00:00<00:00, 11.7MB/s]

Upload an audio file with a spoken engineering question, or click Cancel to use a sample clip.

No file chosen

No file uploaded -- using auto-generated sample audio: /content/sample_query.mp3

```
[transformers] Using custom `forced_decoder_ids` from the (generation) config. This is deprecated in favor of the `task` and `language` fl
[transformers] Transcription using a multilingual Whisper will default to language detection followed by transcription instead of translat
[transformers] A custom logits processor of type <class 'transformers.generation.logits_process.SuppressTokensLogitsProcessor'> has been p
[transformers] A custom logits processor of type <class 'transformers.generation.logits_process.SuppressTokensAtBeginLogitsProcessor'> has
[transformers] Ignoring clean_up_tokenization_spaces=True for BPE tokenizer WhisperTokenizer. The clean_up_tokenization post-processing st
```

Transcribed text: What is the maximum load a steel truss bridge can safely carry?

Lab 30 — Text-to-Speech for Engineering Text

Goal: Convert written engineering text into natural-sounding speech using a pre-trained TTS model. `gTTS` (Google's Text-to-Speech) is used here since it needs no local model download and works reliably in Colab out of the box.

```
from gtts import gTTS
from IPython.display import Audio

engineering_text = (
    "Reinforced concrete combines the compressive strength of concrete with the "
    "tensile strength of embedded steel bars, allowing structures such as bridges "
    "and multi-storey buildings to resist much larger loads than plain concrete alone."
)

tts_path = "/content/engineering_speech.mp3"
gTTS(text=engineering_text, lang="en").save(tts_path)
print(f"Saved: {tts_path}")
# gTTS also supports other languages, e.g. lang='ta' for Tamil or lang='hi' for Hindi.
Audio(tts_path)
```

Saved: /content/engineering_speech.mp3

0:00 / 0:16

Lab 31 — Summarizing a Lengthy Engineering Document

Goal: Summarize a long document that's too big to fit in one prompt, using a map-reduce strategy: split into chunks (reusing `RecursiveCharacterTextSplitter` from the UNIT III RAG labs), summarize each chunk, then combine those partial summaries into one final summary with the LLM.

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

long_document = """Structural health monitoring (SHM) is the process of implementing a
damage detection strategy for civil, mechanical, and aerospace engineering structures.
Traditionally, engineers relied on periodic visual inspections to check for cracks,
corrosion, or fatigue in bridges, buildings, and machinery. Visual inspection is slow,
subjective, and often misses damage that is not visible on the surface, such as internal
delamination in composite materials or corrosion inside a sealed steel member.

Modern SHM systems instead embed a network of sensors -- accelerometers, strain gauges,
fiber-optic sensors, and acoustic emission sensors -- directly into or onto a structure.
These sensors continuously record how the structure responds to everyday loads such as
traffic, wind, and temperature changes. A sudden change in a structure's vibration
frequency, for example, can indicate that stiffness has been lost somewhere, which is
often an early sign of cracking or loosening connections long before the damage would be
visible to the naked eye.
```

The raw sensor data by itself is not very useful; the real value comes from the signal processing and machine learning pipelines built on top of it. Techniques such as Fourier and wavelet transforms convert raw vibration signals into the frequency domain, making it easier to spot the small shifts that indicate developing damage. More recently, machine learning models -- including convolutional neural networks trained on vibration spectrograms -- have been used to automatically classify the type and severity of damage without requiring a human to manually inspect every sensor reading.

One of the most promising recent developments is combining SHM sensor networks with autonomous drones. A drone equipped with a camera and onboard AI can fly a preset inspection route around a bridge or tower, capturing high-resolution images and using computer vision models to flag visible cracks or corrosion, while the embedded sensor network simultaneously tracks internal structural changes. Combining these two data sources gives engineers a much more complete picture of a structure's health than either method alone.

Despite these advances, SHM still faces practical challenges: sensors need reliable power and wireless connectivity over the lifetime of a structure (which can be 50-100 years for major bridges), and false alarms from environmental noise, such as temperature expansion, must be carefully filtered out so that engineers are not overwhelmed with unnecessary alerts. Ongoing research is focused on making SHM systems more energy-efficient, more robust to environmental noise, and cheap enough to deploy at scale across an entire country's aging infrastructure rather than only on a handful of flagship structures."""

```
def summarize_document(text, chunk_size=900):
    splitter = RecursiveCharacterTextSplitter(chunk_size=chunk_size, chunk_overlap=100)
    parts = splitter.split_text(text)
    print(f"Document split into {len(parts)} chunk(s) for summarization.")

    if len(parts) == 1:
        return ask_llm(f"Summarize this engineering text in 4-5 sentences:\n\n{parts[0]}")

    partial_summaries = []
    for i, part in enumerate(parts):
        s = ask_llm(f"Summarize this excerpt in 2-3 sentences, keeping the key technical facts:\n\n{part}")
        partial_summaries.append(s)
        print(f" chunk {i + 1}/{len(parts)} summarized")

    combined = "\n".join(partial_summaries)
    return ask_llm(f"Combine these partial summaries into one coherent 5-7 sentence summary:\n\n{combined}")

# Optional: replace `long_document` above with your own text, e.g.:
# from pypdf import PdfReader
# long_document = "\n".join(p.extract_text() or "" for p in PdfReader("/content/drive/MyDrive/report.pdf").pages)

summary = summarize_document(long_document)
print("\nFinal summary:\n", summary)
```

```
Document split into 5 chunk(s) for summarization.
chunk 1/5 summarized
chunk 2/5 summarized
chunk 3/5 summarized
chunk 4/5 summarized
chunk 5/5 summarized
```

```
Final summary:
Structural health monitoring (SHM) is a systematic, data-driven approach that goes beyond periodic visual inspections to detect hidden da
```

✓ Lab 32 — English → Indian Language Machine Translation

Goal: Translate an engineering sentence from English into Tamil using a dedicated pre-trained translation model, `Helsinki-NLP/opus-mt-en-dra` (English → Dravidian languages; Tamil is selected with the `>>tam<<` target-language tag). If that specific model can't be loaded for any reason, the cell falls back to asking the Groq LLM to translate instead, so the lab still produces a result.

For Hindi instead of Tamil: since Hindi isn't a Dravidian language, use `Helsinki-NLP/opus-mt-en-hi` directly and drop the `>>tam<<` tag.

```
english_text = "Reinforced concrete structures must be designed to resist both tension and compression forces safely."
target_lang_tag = "tam" # Tamil

try:
    from transformers import MarianMTModel, MarianTokenizer

    mt_model_name = "Helsinki-NLP/opus-mt-en-dra"
    mt_tokenizer = MarianTokenizer.from_pretrained(mt_model_name)
    mt_model = MarianMTModel.from_pretrained(mt_model_name)

    tagged_text = f">>{target_lang_tag}<< {english_text}"
    batch = mt_tokenizer([tagged_text], return_tensors="pt", padding=True)
    generated = mt_model.generate(**batch)
    translation = mt_tokenizer.decode(generated[0], skip_special_tokens=True)
```

```
print(f"Translation model used: {mt_model_name} (target: {target_lang_tag})")
except Exception as e:
    print(f"Dedicated translation model unavailable ({e}); falling back to the Groq LLM for translation.")
    translation = ask_llm(
        f"Translate the following engineering sentence into Tamil. Reply with only the translation:\n\n{english_text}"
    )

print("\nEnglish:  ", english_text)
print("Translated:", translation)

tokenizer_config.json: 100%          44.0/44.0 [00:00<00:00, 4.03kB/s]
source.spm: 100%                    818k/818k [00:00<00:00, 17.8MB/s]
target.spm: 100%                    1.17M/1.17M [00:00<00:00, 50.1MB/s]
vocab.json: 100%                    2.34M/2.34M [00:00<00:00, 63.8MB/s]
/usr/local/lib/python3.13/dist-packages/transformers/models/marian/tokenization_marian.py:176: UserWarning: Recommended: pip install sacremoses.
  warnings.warn("Recommended: pip install sacremoses.")
config.json: 100%                    1.40k/1.40k [00:00<00:00, 135kB/s]
pytorch_model.bin: 100%              308M/308M [00:02<00:00, 201MB/s]
Loading weights: 100%               258/258 [00:00<00:00, 7831.82it/s]
model.safetensors: 44%              136M/308M [00:02<00:02, 67.6MB/s]
generation_config.json: 100%         293/293 [00:00<00:00, 21.3kB/s]
Translation model used: Helsinki-NLP/opus-mt-en-dra (target: tam)

English:    Reinforced concrete structures must be designed to resist both tension and compression forces safely.
Translated: ரீன்டர்ட் கான்க்ரீட் அமைப்புகள், அழுத்தத்தையும் அடித்தத்தையும் பத்திரமாக எதிர்க்கும்படி வடிவமைக்கப்பட வேண்டும்.
```

Lab 33 — AI-Based Resume Screening Against a Job Description

Goal: Embed a job description and a set of resumes with the same sentence-transformers model from Labs 15-19, rank resumes by cosine similarity to the job description, and ask the LLM to justify the top match in plain language.

```
job_description = """We are hiring a Junior Mechanical Design Engineer. Requirements: strong
fundamentals in CAD (SolidWorks or AutoCAD), knowledge of thermodynamics and fluid mechanics,
hands-on experience with a mechanical design or robotics project, familiarity with GD&T and
manufacturing processes, and good communication skills for cross-functional collaboration."""

resume_texts = {
    "resume_1.txt": """Aditi Rao - Mechanical Engineering graduate. Proficient in SolidWorks and
AutoCAD with two semester-long CAD design projects. Strong coursework in thermodynamics and
fluid mechanics. Built a robotic arm for a college competition. Familiar with GD&T standards
and CNC machining basics. Comfortable presenting technical work to non-technical stakeholders.""",
    "resume_2.txt": """Karan Mehta - Computer Science graduate with a minor in electronics.
Skilled in Python, Java, and web development. Built a mobile app for campus event management.
No formal mechanical design or CAD coursework. Interested in software engineering roles.""",
    "resume_3.txt": """Sana Iyer - Mechanical Engineering graduate, robotics club lead. Designed
and manufactured a quadruped robot chassis using SolidWorks, iterated through GD&T-compliant
tolerancing, and ran fluid mechanics simulations for a college cooling-system project. Some
public speaking experience from club leadership.""",
}

# To screen your own resumes instead: load .txt/.pdf files from Drive into resume_texts,
# reusing the same load_text() helper style from the UNIT III RAG labs.

jd_embedding = embedder.encode(job_description, convert_to_tensor=True)
scored = []
for name, text in resume_texts.items():
    emb = embedder.encode(text, convert_to_tensor=True)
    score = util.cos_sim(jd_embedding, emb).item()
    scored.append((name, score, text))
scored.sort(key=lambda x: -x[1])

print("Ranked candidates:\n")
for rank, (name, score, _) in enumerate(scored, start=1):
    print(f"#{rank} {name} -- match score: {score:.4f}")

top_name, top_score, top_text = scored[0]
justification = ask_llm(
    f"Job description:\n{job_description}\n\nCandidate resume:\n{top_text}\n\n"
    f"In 2-3 sentences, explain why this candidate is a strong match for the role."
)
print(f"\nWhy {top_name} ranks #1:\n{justification}")
```

```
Ranked candidates:

#1 resume_1.txt -- match score: 0.5566
#2 resume_3.txt -- match score: 0.3758
#3 resume_2.txt -- match score: 0.2964
```


Lab 34 — AI Research Assistant (Overview, Keywords, Summary)

Goal: Given a research topic, generate an LLM overview and abstract-style summary, and separately extract keywords using an actual NLP technique -- TF-IDF over the generated sentences -- rather than just asking the LLM to list keywords.

```
import re
from sklearn.feature_extraction.text import TfidfVectorizer

def research_assistant(topic, num_keywords=8):
    overview = ask_llm(
        f"Give a concise, factual overview (4-6 sentences) of the engineering research topic "
        f"'{topic}'. Focus on what it is, why it matters, and one current trend."
    )
    summary = ask_llm(
        f"Write a 2-sentence abstract-style summary of '{topic}' suitable for a research paper introduction."
    )

    # TF-IDF keyword extraction: treat each sentence as a mini-document so IDF is meaningful,
    # instead of running TF-IDF on a single blob of text (which degenerates to plain term
    # frequency).
    corpus_text = overview + " " + summary
    sentences = [s for s in re.split(r'(?<=[!?.])\s+', corpus_text.strip()) if s]
    vectorizer = TfidfVectorizer(stop_words="english", max_features=30)
    tfidf_matrix = vectorizer.fit_transform(sentences)
    scores = tfidf_matrix.sum(axis=0).A1
    terms = vectorizer.get_feature_names_out()
    ranked_terms = sorted(zip(terms, scores), key=lambda x: -x[1])
    keywords = [t for t, s in ranked_terms[:num_keywords] if s > 0]

    return overview, summary, keywords

topic = "autonomous drone-based structural inspection of bridges"
overview, summary, keywords = research_assistant(topic)

print("Topic:", topic)
print("\nOverview:\n", overview)
print("\nSummary:\n", summary)
print("\nKeywords (TF-IDF):", " ", ".join(keywords))
```

Topic: autonomous drone-based structural inspection of bridges

Overview:
Autonomous drone-based structural inspection of bridges employs self-navigating UAVs equipped with high-resolution cameras, LiDAR, and ot

Summary:
This study presents an autonomous drone platform that fuses high-resolution visual imaging, LiDAR scanning, and deep-learning-based defect

Keywords (TF-IDF): inspection, bridge, detection, time, data, drone, safety, continuous

Summary

Lab	Concept	Key technique
25	College FAQ chatbot	Prompt-grounded LLM (Groq)
26	Technical-support chatbot	Tokenization + stopword filtering -> focused LLM prompt
27	Text-to-image	Diffusion model (<code>stabilityai/sd-turbo</code>)
28	Prompt-variation comparison	Same diffusion model, style-keyword ablation
29	Speech-to-Text	Whisper ASR (<code>openai/whisper-base</code>)
30	Text-to-Speech	gTTS
31	Long-document summarization	Chunking (<code>RecursiveCharacterTextSplitter</code>) + map-reduce LLM summarization
32	Machine translation	Dedicated NMT model (<code>opus-mt-en-dra</code>), LLM fallback
33	Resume screening	Sentence embeddings + cosine similarity ranking
34	Research assistant	LLM generation + TF-IDF keyword extraction

Notes on reliability

- GPU vs CPU:** Labs 27-28 will run on CPU but are much faster with a GPU runtime enabled (`Runtime -> Change runtime type -> GPU`).
- Groq model:** `LLM_MODEL` is set to `openai/gpt-oss-120b` in the Setup cell, since `llama-3.3-70b-versatile` and `llama-3.1-8b-instant` were both retired by Groq. If you see a `NotFoundError: model_not_found` anywhere, check that line first.
- No file/mic required:** Labs 29 and 31 both work out of the box with auto-generated fallback content if you don't upload your own audio/document -- update the relevant variable (`audio_path` upload, or `long_document`) to use your own.

- **Torch on Colab:** the Setup cell installs `torch` without `--upgrade` on purpose, so it doesn't overwrite Colab's pre-installed, GPU-